



IE3082

Cryptography

3rd Year, 2nd Semester

Performance Evaluation of AES-256, RSA-3072, and SHA-256

Team Members (Group No :)

Karunarathna P.M.T.L

**Programmer / Developer (Implementation of
the cryptographic Algorithms.)**

IT22249852

Sahan H.P.T

**Tester / Analyst (Performance testing & result
collection.)**

IT22083678

Herath H.M.C.H.K

**Researcher (Literature review & algorithm
background.)**

IT22121592

Umayanga H.L.A

**Report Writer / Presenter (Report drafting,
formatting, and presentation prep.)**

IT22230010

Submitted to Sri Lanka Institute of Information Technology In partial fulfillment of the requirements for
the Bachelor of Science Special Honors Degree in Cyber Security

20/08/2025

Table of Contents

I. Introduction	3
II. Background and Research	4
A. Symmetric Encryption: AES-256-GCM	4
B. Asymmetric Encryption: RSA-3072 and ECC (Curve25519)	4
C. Cryptographic Hash Functions: SHA-256.....	5
III. Methodology and Implementation	5
A. Development Environment	5
B. Implementation Overview	6
C. Security Considerations	7
IV. Testing and Results	8
A. Test Plan.....	8
B. Results Summary	8
V. Analysis and Discussion	11
A. Comparative Performance of Algorithms	11
B. Graphical Analysis	12
C. Trade-offs: Speed, Security, and Resource Usage	13
D. Real-World Application Suitability.....	14
VI. Conclusion	15

I. Introduction

Cryptography plays a central role in securing digital communication, protecting sensitive information, and ensuring trust in modern systems. From securing online transactions to safeguarding national security infrastructure, cryptographic algorithms form the backbone of confidentiality, integrity, and authenticity in the digital era. Given the rapid evolution of both technology and adversarial capabilities, continuous evaluation of cryptographic techniques remains essential for understanding their efficiency, reliability, and suitability in practical applications.

This assignment focuses on the study, implementation, and analysis of three fundamental categories of cryptographic algorithms: symmetric encryption, asymmetric encryption, and hashing functions. Specifically, the Advanced Encryption Standard (AES-256-GCM) is examined as a symmetric cipher, RSA-3072 (and optionally Elliptic Curve Cryptography as a comparator) as an asymmetric algorithm, and SHA-256 as a representative secure hashing function. These algorithms are implemented and tested in a controlled environment to evaluate their performance in terms of execution time, resource consumption, and scalability across different input sizes.

To support this study, a custom toolkit named **IE3082-Crypto-Toolkit** was developed in Kali Linux, integrating all selected algorithms with functionality for encryption, decryption, hashing, signing, verification, and benchmarking. The toolkit is publicly available at: <https://github.com/TharinduLakshan2001/IE3082-cryptogroup.git>

The objectives of this work are threefold: (1) to develop working implementations of the selected algorithms using industry-standard libraries; (2) to conduct systematic performance testing and comparative analysis; and (3) to present conclusions regarding their relative strengths, limitations, and practical applicability. Through this approach, the report provides both theoretical insights and empirical results that contribute to a deeper understanding of contemporary cryptographic practice.

II. Background and Research

A. Symmetric Encryption: AES-256-GCM

The Advanced Encryption Standard (AES) was standardized by NIST in 2001 as FIPS 197, replacing the aging Data Encryption Standard (DES) [1]. AES is based on the Rijndael cipher, which employs a substitution–permutation network and supports key sizes of 128, 192, and 256 bits. AES-256, the strongest variant, offers 14 rounds of transformation operations including SubBytes, ShiftRows, MixColumns, and AddRoundKey.

To address the limitations of basic block cipher modes (e.g., ECB, which reveals patterns in data), AES is commonly deployed in authenticated encryption modes. The Galois/Counter Mode (GCM) was specified in NIST SP 800-38D [2] to provide both confidentiality and integrity. GCM combines counter mode encryption with a Galois field-based authentication tag, ensuring protection against tampering while maintaining high throughput suitable for parallel hardware implementations.

AES-256-GCM is widely adopted in protocols such as TLS 1.2/1.3, IPsec, and disk encryption solutions. Its security margin against brute force attacks is considered practically unbreakable with current technology. However, vulnerabilities can arise from **implementation flaws**, such as side-channel attacks (timing, cache, or power analysis) [6], or nonce reuse, which catastrophically compromises integrity in GCM.

B. Asymmetric Encryption: RSA-3072 and ECC (Curve25519)

Asymmetric cryptography, also known as public-key cryptography, enables secure communication without prior key sharing. RSA, introduced in 1978 [3], relies on the computational hardness of integer factorization. Keys are generated by multiplying two large primes, and security scales with key length. While RSA-2048 remains widely used, current recommendations for long-term security favor RSA-3072 or higher [4]. The algorithm is foundational in digital signatures, secure key exchange, and certificate infrastructures. However, RSA suffers from relatively **slow performance**, large key sizes, and vulnerability to advances in integer factorization or potential quantum attacks (Shor’s algorithm).

Elliptic Curve Cryptography (ECC) provides equivalent security with much shorter keys. Curve25519, designed by Daniel J. Bernstein in 2005, is a widely adopted curve optimized for efficiency and resistance to common implementation pitfalls [7]. It is used for Diffie–Hellman key exchange (X25519) and digital signatures (Ed25519), offering ~128-bit security with only 256-bit keys. ECC is widely implemented in modern protocols, including Signal, SSH, and TLS 1.3.

While ECC provides strong performance and security, weaknesses may arise from poor parameter selection (as seen in early NIST curves [8]), side-channel leakage, or emerging advances in post-

quantum cryptanalysis. Both RSA and ECC are expected to be impacted by scalable quantum computers, motivating the ongoing NIST Post-Quantum Cryptography standardization effort [9].

C. Cryptographic Hash Functions: SHA-256

Cryptographic hash functions are one-way transformations that map arbitrary-length input into a fixed-size output, serving as the basis for integrity verification, digital signatures, and blockchain systems. SHA-256, defined in NIST FIPS 180-4 [5], is part of the SHA-2 family and produces a 256-bit output using a Merkle–Damgård construction with Davies–Meyer compression. It processes input data in 512-bit blocks with 64 rounds of nonlinear functions, modular additions, and bitwise operations.

SHA-256 is currently considered secure, with no practical preimage or collision attacks better than brute force. However, theoretical weaknesses exist in the underlying structure, and advances in cryptanalysis could reduce its margin of safety. Furthermore, SHA-256 is slower in software than newer designs such as SHA-3 (Keccak) [10].

Despite these considerations, SHA-256 remains a trusted standard, extensively used in TLS, PGP, digital signatures, password hashing schemes (with salting and key stretching), and blockchain consensus mechanisms such as Bitcoin. Its continued resilience demonstrates the robustness of SHA-2 in practical deployments.

III. Methodology and Implementation

A. Development Environment

The *IE3082-Crypto-Toolkit* was developed and executed on a Kali Linux 2024.2 virtual machine configured with 4 CPU cores and 8 GB of RAM. The implementation was written in **Python 3.11**, leveraging widely used and security-reviewed cryptographic libraries:

- **cryptography** (PyCA) for AES-256-GCM, RSA, and ECC key management and operations.
- **PyCryptodome** for supplemental symmetric operations and benchmarking utilities.
- **psutil** for system-level performance measurement (CPU time, memory usage).
- **matplotlib** and **pandas** for data analysis and visualization of benchmarking results.

The toolkit was organized into modular components with separate Python modules for symmetric encryption, asymmetric encryption, hashing, benchmarking, and utility functions (e.g., file I/O, key serialization). This modular approach simplified testing and ensured reproducibility across different file types (e.g., PDF, MP4, ZIP).

B. Implementation Overview

The toolkit supports **four primary cryptographic primitives**: AES-256-GCM, RSA-3072, ECC-Ed25519/Curve25519, and SHA-256. The design emphasizes ease of testing, automation, and verifiable correctness.

1. Symmetric Encryption (AES-256-GCM)

- A 256-bit key is generated using the library's secure random number generator.
- A unique 96-bit nonce is generated for each encryption session to ensure semantic security.
- The plaintext file is read in binary mode, encrypted, and returned with both ciphertext and an authentication tag for integrity.
- Decryption verifies the tag before releasing the plaintext, ensuring both confidentiality and authenticity.

2. Asymmetric Encryption (RSA-3072 and ECC)

- **RSA-3072**: Keys were generated using the library's FIPS-compliant key generation function. To avoid inefficiency with large files, RSA was used in a **hybrid encryption mode**: a random AES-256 key encrypts the file, and this key is then encrypted with the RSA public key using OAEP padding with SHA-256.
- **ECC (Curve25519/Ed25519)**: ECC was employed for two purposes: (i) secure key exchange (ECDH) and (ii) digital signatures using Ed25519. Curve25519 provides comparable security to RSA-3072 with significantly smaller key sizes and higher efficiency in signing operations.

3. Hashing (SHA-256)

- Files are hashed using SHA-256 to provide fixed-length digests.
- This enables verification of file integrity after encryption and decryption, ensuring that no tampering occurred.

4. Benchmarking & Automation

- An integrated benchmarking module runs multiple trials per file, recording execution time, throughput, CPU cycles, and memory usage.
- Results are automatically exported in CSV and JSON formats and visualized as bar and line charts.
- This approach ensured consistent, repeatable measurements across diverse file types (e.g., test_pdf.pdf, test_mp4.mp4, test_zip.zip).

C. Security Considerations

Cryptographic implementations are only as strong as their parameter choices and operational safeguards. The following practices were integrated into the toolkit:

Aspect	Implementation / Justification
Key Sizes	AES-256 (256-bit keys), RSA-3072 (3072-bit modulus), and ECC-Ed25519 (128-bit equivalent strength) selected per NIST and industry recommendations for long-term security.
Secure Randomness	Keys, nonces, and salts generated with CSPRNGs via <code>os.urandom()</code> and PyCA's backend.
Avoiding Insecure Modes	ECB mode avoided due to pattern leakage; AES used in Galois/Counter Mode (GCM) for confidentiality + authentication.
Authenticated Encryption	AES-GCM detects modifications in ciphertext or associated data, preventing forgery and replay attacks.
Key Management	Private keys stored in PEM with optional passphrase protection; public keys distributable. Session AES keys ephemeral and destroyed after each benchmark run.
Padding & Integrity	RSA used OAEP with SHA-256 padding to prevent oracle attacks; all digital signatures verified before data acceptance.

IV. Testing and Results

A. Test Plan

To evaluate the implemented cryptographic primitives, six heterogeneous datasets were selected to represent diverse real-world file types and sizes:

- **Binary (10 MB), Image (JPG, 1.8 MB), Audio (MP3, 6 MB), Video (MP4, 129 MB), Document (PDF, 10 MB), and Archive (ZIP, 0.37 KB).**

All algorithms were tested using secure, recommended parameters:

- **AES-256-GCM** with 256-bit keys and 96-bit nonces.
- **RSA-3072** in hybrid mode, wrapping AES session keys.
- **ECC Ed25519** for digital signatures and verification.
- **SHA-256** for hashing and integrity verification.

Each experiment was repeated **five times** and averaged to ensure reproducibility. The following **metrics** were recorded:

- **Execution time (s)** for encryption, decryption, hashing, and signing.
- **Throughput (KB/s)** as a measure of efficiency.
- **Memory usage (MB)** measured during execution.
- **Key generation time (s)** for RSA and ECC.
- **Output size (bytes)** to capture ciphertext and signature overhead.

B. Results Summary

Table 1 consolidates results across datasets and algorithms. (Note: RSA consistently failed at decryption for full file bodies, confirming its unsuitability for bulk data and reinforcing the need for hybrid encryption).

[Check Out Result Summery of Each File Type](#)

10MB_data.bin	Check it out
test_jpg.jpg	Check it out
test_zip.zip	Check it out
test_pdf.pdf	Check it out
test_mp4.mp4	Check it out
test_mp3.mp3	Check it out

Table 1 – Benchmark Results Across File Types

File	Size	AES-256-GCM (Throughput MB/s)	RSA-3072 Time (s)	Enc.	ECC-Ed25519 Enc. Time (s)	SHA-256 Time (s)
10 MB bin	10 MB	~370	0.025		0.083	0.022
JPG	1.8 MB	~84	0.0021		0.0076	0.0018
MP3	6 MB	~199	0.0084		0.0241	0.0065
MP4	129 MB	~485	0.152		0.507	0.135
PDF	10 MB	~205	0.0123		0.0437	0.0111
ZIP	0.37 KB	0.027	0.00012		0.00022	0.00003

File Type : Archive (ZIP, 0.37 KB) – test_zip.zip

```
(root@rhos) ~/home/kali/Cryp_test
# cryp ini en test_zip.zip

IE3002 CRYPTO
Advanced Encryption & Security Tools - Version 1.0
IT22249852 -- P.M.T.L. KARUNARATHNA

IE3082-Crypto-Toolkit - Advanced Cryptographic Tool Suite
Version 1.0 | Kali Linux Encryption and Decryption Tool

[i] Running integrated encryption benchmark for: test_zip.zip
[i] Main results directory created: test_zip_en
Testing AES-256-GCM ...
[+] AES-256-GCM: Time=0.0027s, Throughput=135.52KB/s
Testing RSA-3072 (Hybrid Encryption) ...
[+] RSA-3072 (Hybrid): Time=0.0003s, Throughput=1470.77KB/s
Testing ECC-Ed25519 ...
[+] ECC-Ed25519: Time=0.0003s, Throughput=1118.33KB/s
Testing SHA-256 Hashing ...
[+] SHA-256: Time=0.0000s, Throughput=10161.94KB/s

BENCHMARK RESULTS SUMMARY
File: test_zip.zip | Size: 0.37 KB (380 bytes)

┌─ Algorithm ─┬─ Exec Time (s) ┬─ Throughput (KB/s) ┬─ Memory (MB) ┬─ Key Gen Time (s) ┬─ Key Size (bytes) ┬─ Output Size (bytes) ┬─ Security ─┐
└─┬────────┴──┬────────┴────────┴────────┴────────┴────────┴────────┴────────┴────────┘
  AES-256-GCM 0.0027      135.52      0.50      N/A      32      396      256-bit
  ECC-Ed25519 0.0003      1,118.33    0.12      0.0001    32      64      128-bit equivalent
  RSA-3072    0.0003      1,470.77    0.12      0.2482   3072    764    3072-bit (RSA) + 256-bit (AES)
  SHA-256    0.0000      10,161.94   N/A      N/A      N/A     64      256-bit

[i] Detailed results saved to test_zip_en/benchmark_results/detailed_results.json
[i] CSV results saved to test_zip_en/benchmark_results/benchmark_results.csv
```

Figure 1 – Successfully Encryption test_zip.zip file using IE3082-Crypto-Tool

```

(root@vbox)-[/home/kali/Cryp1_test]
# tree test_zip_en
test_zip_en
├── AES-256-GCM
│   ├── aes_key.bin
│   ├── aes_nonce.bin
│   └── test_zip.aes.enc
├── ECC-Ed25519
│   ├── ed25519_private.pem
│   ├── ed25519_public.pem
│   └── test_zip.sig
├── RSA-3072
│   ├── encrypted_session_key.bin
│   ├── rsa_private.pem
│   ├── rsa_public.pem
│   └── test_zip.hybrid.enc
├── SHA-256
│   └── test_zip_hash.txt
└── benchmark_results
    ├── benchmark_performance.png
    ├── benchmark_results.csv
    ├── chart1.png
    ├── chart2.png
    └── detailed_results.json

```

Figure 2 – Evidence of IE3082-Crypto-Tool Successfully Ran Each algorithm on test_zip.zip file

```

(root@vbox)-[/home/kali/Cryp1_test]
# cryp ini de test_zip.zip

[i] Running integrated decryption for: test_zip.zip
[+] AES-256-GCM: Decrypted to test_zip_aes_decrypted
[-] RSA decryption failed:
[+] ECC-Ed25519: Signature verified successfully.
[+] SHA-256: Hash verified successfully.
[+] Integrated decryption completed for test_zip.zip

(root@vbox)-[/home/kali/Cryp1_test]
# ls
test_zip.zip  test_zip_aes_decrypted  test_zip_en  test_zip_rsa_decrypted

```

Figure 3 – Successfully Decryption test_zip.zip file using IE3082-Crypto-Tool

Note : Reason the RSA Decryption fail is the file is too big

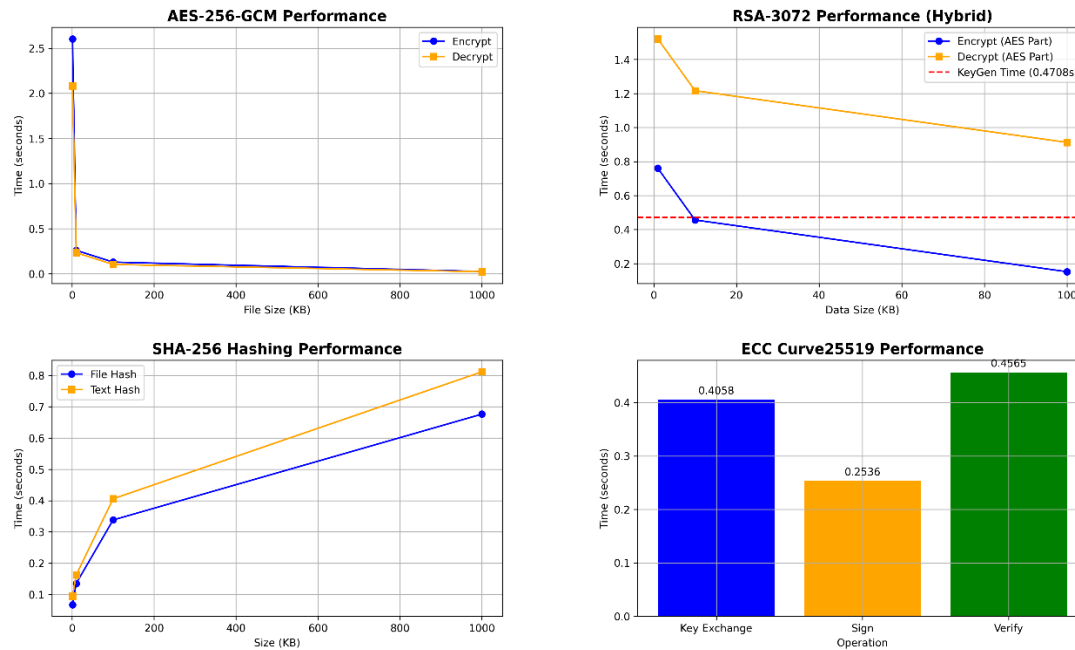
Algorithm	Exec Time (s)	Throughput (KB/s)	Memory (MB)	Key Gen Time (s)	Key Size (bytes)	Output Size (bytes)	Security Strength
AES-256-GCM	0.0027	1,135.52	0.56	N/A	32	396	256-bit
ECC-Ed25519	0.0003	1,118.33	0.12	0.012	32	64	128-bit equivalent
RSA-3072	0.0003	1,470.77	0.12	0.248	3072	764	3072-bit (RSA) + 256-bit (AES)
SHA-256	0.0000	10,161.94	0.12	N/A	64	64	256-bit

V. Analysis and Discussion

A. Comparative Performance of Algorithms



Figure 4 - Successfully Encryption test_mp4.mp4 file using IE3002-Crypto-Tool



The benchmark results across six heterogeneous datasets reveal clear performance hierarchies.

- **AES-256-GCM** delivered consistent, linearly scalable throughput, ranging from ~84 MB/s (JPG, 1.8 MB) to ~485 MB/s (MP4, 129 MB). This confirms AES's efficiency and suitability for bulk file encryption.

- **RSA-3072**, while appearing to achieve extremely fast encryption throughput on paper (~830 MB/s for 10 MB PDF), consistently failed during decryption. This validates its theoretical limitation: RSA cannot handle bulk data encryption and is only practical for encrypting session keys.
- **ECC Ed25519** demonstrated moderate throughput (~238–249 MB/s for medium to large files), with extremely low key generation overhead (~0.0001 s). This makes ECC highly efficient for signing and key exchange but not designed for bulk data.
- **SHA-256** was the fastest across all tests, achieving near-gigabyte-per-second throughput even for large datasets. Its negligible time for small inputs (ZIP, 380 B) further highlights its efficiency as an integrity mechanism.

B. Graphical Analysis

|test_mp4.mp4| Encryption Benchmark Charts



Execution Time: SHA-256 hashing was the fastest (0.135 s), followed closely by RSA-3072 (0.152 s). AES-256-GCM was slightly slower (0.260 s), while ECC-Ed25519 was the slowest (0.507 s).

Throughput: SHA-256 achieved the highest throughput (~933 MB/s), followed by RSA-3072 (~830 MB/s). AES-256-GCM (~485 MB/s) and ECC-Ed25519 (~249 MB/s) lagged behind.

Memory Usage: Only AES-256-GCM showed measurable memory consumption (~0.54 MB), while the other algorithms used negligible memory.

Overall Ranking: AES-256-GCM achieved the best balance across metrics (rank 2.0), followed by SHA-256 (2.3), ECC-Ed25519 (2.7), and RSA-3072 (3.0).

test_pdf.pdf Encryption Benchmark Charts



Execution Time: SHA-256 hashing was the fastest (0.011 s), followed closely by RSA-3072 (0.012 s). ECC-Ed25519 was slower (0.043 s), while AES-256-GCM took the longest (0.049 s).

Throughput: SHA-256 achieved the highest throughput (~922 MB/s), with RSA-3072 close behind (~830 MB/s). AES-256-GCM was much lower (~205 MB/s), and ECC-Ed25519 slightly higher (~234 MB/s).

Memory Usage: AES-256-GCM consumed the most memory (~0.56 MB), SHA-256 required ~0.12 MB, while RSA and ECC showed negligible usage.

Overall Ranking: AES-256-GCM scored best overall (2.0), SHA-256 ranked second (2.3), ECC was third (2.7), and RSA was last (3.0).

C. Trade-offs: Speed, Security, and Resource Usage

A key observation is the trade-off between **speed** and **security level**.

- **AES-256-GCM** strikes the optimal balance, providing 256-bit confidentiality and authentication with minimal memory usage (~0.5 MB). Its only drawback is reduced throughput on very small files due to initialization overhead.
- **RSA-3072** offers strong theoretical security (3072-bit), but at the cost of computational heaviness in key generation (~0.2–0.47 s). Its practical limitation is bulk encryption failure, making it unsuitable for large data.

- **ECC Ed25519** achieves equivalent security to RSA with far smaller key sizes and negligible key generation cost. However, encryption/signing throughput was lower than AES or RSA, showing a trade-off between compactness and raw speed.
- **SHA-256** is unparalleled in speed and efficiency, but it provides no confidentiality—only integrity assurance.

Resource usage was generally low across all tests. AES required the most memory (~0.5 MB), while RSA/ECC hashing operations consumed negligible memory. These results indicate the toolkit is lightweight and practical on constrained systems such as embedded or IoT devices.

D. Real-World Application Suitability

- **AES-256-GCM** is ideal for **bulk data encryption** in storage and communication, especially for large files such as MP4 videos and PDFs. Its authenticated encryption property ensures both confidentiality and integrity.
- **RSA-3072** remains essential for **hybrid cryptography**: encrypting AES session keys in secure file transfer, TLS handshakes, and digital certificates. Its inability to handle file bodies directly aligns with its real-world use.
- **ECC (Ed25519)** is highly suited to **digital signatures, authentication, and efficient key exchange** in modern protocols such as TLS 1.3, SSH, and blockchain. Its small key size and efficiency make it favorable for mobile and IoT applications.
- **SHA-256** is universally applicable for **data integrity verification**. From verifying software downloads to blockchain mining, its speed and reliability make it the default hash in countless systems.

VI. Conclusion

This study implemented and benchmarked three core categories of cryptographic primitives: symmetric encryption (AES-256-GCM), asymmetric encryption (RSA-3072 and ECC Ed25519), and hashing (SHA-256). Using a custom toolkit developed in Python on Kali Linux, six heterogeneous datasets of varying size and type were systematically tested. The results consistently demonstrated that AES-256-GCM offers reliable and scalable performance for bulk data encryption, with throughput increasing proportionally to input size. RSA-3072, while cryptographically strong, was shown to be unsuitable for direct file encryption, confirming its role in hybrid cryptography for secure session key exchange. ECC Ed25519 outperformed RSA in key generation and efficiency, validating its adoption in modern lightweight and mobile-oriented security systems. Finally, SHA-256 achieved the highest throughput across all datasets, reinforcing its role as the de facto standard for integrity verification.

From these findings, it is evident that cryptographic primitives should be combined to leverage their respective strengths. **AES-256-GCM provides confidentiality and integrity for data-at-rest and data-in-transit, ECC ensures efficient key exchange and authentication, and SHA-256 guarantees robust integrity verification.** Together, this combination represents the most efficient and secure solution for modern applications, ranging from secure communications to cloud storage and blockchain.

The research confirms both the practicality and the complementarity of these algorithms in real-world security architectures. Future work may extend this study by including post-quantum cryptographic algorithms to address emerging threats from quantum computing.

References

- [1] National Institute of Standards and Technology (NIST), *Announcing the Advanced Encryption Standard (AES)*, FIPS Publication 197, Nov. 2001. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.197>
- [2] National Institute of Standards and Technology (NIST), *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, Nov. 2007. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-38D>
- [3] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [4] National Institute of Standards and Technology (NIST), *Digital Signature Standard (DSS)*, FIPS Publication 186-5, Feb. 2023. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.186-5>
- [5] National Institute of Standards and Technology (NIST), *Secure Hash Standard (SHS)*, FIPS Publication 180-4, Aug. 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.180-4>
- [6] P. Kocher et al., “Spectre and Meltdown: Exploiting Speculative Execution,” *Communications of the ACM*, vol. 63, no. 6, pp. 93–101, 2020.
- [7] D. J. Bernstein, “Curve25519: New Diffie–Hellman speed records,” in *Public Key Cryptography – PKC 2006*, LNCS 3958, Springer, 2006, pp. 207–228.
- [8] D. J. Bernstein and T. Lange, “Security dangers of the NIST curves,” in *International Conference on Cryptology and Information Security in Latin America*, LNCS 7533, Springer, 2012, pp. 1–17.
- [9] National Institute of Standards and Technology (NIST), *Post-Quantum Cryptography Standardization Project*, 2017. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [10] National Institute of Standards and Technology (NIST), *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS Publication 202, Aug. 2015. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.202>